

来源: <https://docs.springframework.org.cn/spring-ai/reference/api/chat-memory.html>
版本: 2.0.1

=====

一、概述

=====

大型语言模型 (LLM) 本身是无状态的，不会保留之前交互的信息。当你想要在多次交互中维持上下文时，这一点就会带来问题。为了解决这个问题，Spring AI 提供了聊天记忆 (Chat Memory) 功能，允许你在与 LLM 的多次交互中存储和检索信息。

ChatMemory 接口抽象了聊天记忆的存储，目前提供以下实现：

- MessageWindowChatMemory (消息窗口聊天记忆)

Spring AI 自动配置 (auto-configuration) 会自动配置一个 ChatMemory Bean，你可以直接注入到应用中。默认使用内存中的存储库 (InMemoryChatMemoryRepository) 和 MessageWindowChatMemory 策略。如果你的应用配置了 Cassandra、JDBC 或 Neo4j 存储库，自动配置会改用对应的存储库。

=====

二、聊天记忆 vs 聊天历史 (关键区别)

=====

这两个概念密切相关但用途不同：

1. 聊天记忆 (Chat Memory)
聊天记忆是为了维持上下文而保留的关键信息。它由 ChatMemory 接口管理，是一种受控的、有选择的存储 —— 为了保持对话连贯，只保留必要的信息。
2. 聊天历史 (Chat History)
聊天历史是用户和模型之间所有交互的完整记录。ChatMemory 并不适合存储完整的聊天历史。如果需要保存完整的对话记录，建议使用更合适的方案 (例如基于 Spring Data 的持久化方案) 来保存完整的交互信息。

简而言之：

- ChatMemory 适合：维持对话上下文，有选择地保留关键信息。
- Chat History 适合：完整记录每一次交互，用于审计、回放、分析等。

=====

三、快速开始

=====

Spring AI 自动配置会自动配置一个 ChatMemory Bean。默认配置使用：

- 存储库：InMemoryChatMemoryRepository（内存存储）
- 记忆策略：MessageWindowChatMemory

如果类路径上存在 Cassandra、JDBC 或 Neo4j 存储库，自动配置将使用它们。

要使用聊天记忆，可以向 ChatClient 添加 MessageChatMemoryAdvisor（或特定于提供者的实现，例如 PromptChatMemoryAdvisor）：

```
ChatMemory chatMemory = ...;

ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
    .build();
```

注意：内存中的 ChatMemory 仓库（InMemoryChatMemoryRepository）不是分布式的，因此在多实例/分布式环境中，不同实例之间的聊天记忆不会共享。生产环境建议使用持久化存储库。

=====

四、关于会话 ID 的重要说明

=====

每一个聊天记忆操作都以会话 ID（conversation ID）为键。每个存储库实现都使用会话 ID 来存储和检索消息。

重要：必须显式提供会话 ID。当你使用 ChatClient 时，可以通过 advisor 参数传入会话 ID：

```
// 为每个用户/会话生成唯一的会话 ID
String conversationId = currentUser.getId() + ":" + httpSession.getId();

String answer = chatClient.prompt()
    .user(userInput)
    .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
    .call()
    .content();
```

在多用户应用中，务必确保会话 ID 的生成方式能够为每个用户的每次对话生成唯一的值。最佳实践是在服务端从当前用户身份和会话信息派生出会话 ID，而不是让客户端直接传入。

五、记忆类型：MessageWindowChatMemory

MessageWindowChatMemory 是最常用的记忆类型。它维护一个消息窗口，默认窗口大小为 20 条消息。当消息数量超过窗口大小时，它会淘汰较旧的消息，但始终保留 SystemMessage（系统消息）。

```
MessageWindowChatMemory memory = MessageWindowChatMemory.builder()
    .chatMemoryRepository(new InMemoryChatMemoryRepository())
    .maxMessages(20)
    .build();
```

工作原理要点：

1. 消息窗口：只保留最近的消息，最多 maxMessages 条。
2. 始终保留 SystemMessage：系统消息不会被淘汰，因为它通常包含重要的指令/上下文。
3. 按轮次（turn）驱逐：
 - 一个 "turn" 从一条 UserMessage 开始，包含其后的所有回复消息、工具调用消息，直到下一个 UserMessage 之前。
 - 当需要淘汰消息时，会整体淘汰一个 turn，而不会从 turn 中间截断。
 - 因此 maxMessages 实际上是消息条数的上限，由于按整轮淘汰，实际保留的消息数可能略少于 maxMessages。
 - 如果 maxMessages 设置得太小，可能会导致整个 turn 被淘汰掉。

六、存储库（ChatMemoryRepository）实现

ChatMemoryRepository 负责实际的存储。Spring AI 提供以下实现：

1. InMemoryChatMemoryRepository（内存）
 - 默认实现，使用 ConcurrentHashMap 存储。
 - 每个会话的消息保存在内存中，应用重启后丢失。
 - 非分布式，不适合多实例部署。
2. JdbcChatMemoryRepository（JDBC + 关系型数据库）
 - 支持持久化到关系型数据库。
 - 支持的数据库：PostgreSQL、MySQL / MariaDB、Microsoft SQL Server、HSQLDB、Oracle。
 - 详见下文 "JDBC 存储库" 章节。

3. CassandraChatMemoryRepository (Cassandra)

- 支持持久化到 Apache Cassandra。

(原文档在此处被截断，以下为 JDBC 部分的关键内容。)

七、JDBC 存储库 (JdbcChatMemoryRepository)

JdbcChatMemoryRepository 使用 JDBC 将聊天记忆持久化到关系型数据库。

注意事项 1: 不支持工具调用消息

- JdbcChatMemoryRepository 目前不支持工具调用消息的存储。
- 带工具调用的 AssistantMessage 以及 ToolResponseMessage 在保存时会被静默过滤掉。
- 如果你的应用使用了工具调用，建议改用 Spring AI Session 配合 JDBC 会话存储 (JDBC Session Storage)。

注意事项 2: 消息检索顺序与时间戳

- 消息按 sequence_id 升序检索。
- 每条消息带有创建时间戳，存储在其元数据中，key 为 CONVERSATION_TS，类型为 java.time.Instant。
- 该时间戳可用于在用户界面 (UI) 中显示消息时间。

配置属性:

- spring.ai.chat.memory.repository.jdbc.initialize-schema
是否初始化 schema。取值: embedded (默认) / always / never。
- spring.ai.chat.memory.repository.jdbc.schema
指定初始化脚本的位置。
- spring.ai.chat.memory.repository.jdbc.platform
指定平台 (数据库平台)，用于选择对应的初始化脚本。

Schema 变更说明:

- 自 Spring AI 2.0.0 起，表结构新增了 sequence_id 列，用于保证消息顺序。
- 从更早版本升级的用户需要手动为表添加该列。

八、常见问题小结

1. LLM 无状态，需要 ChatMemory 来维持上下文。
2. ChatMemory 管理的是"为维持上下文而保留的关键信息"，不是完整历史。
3. 必须显式提供会话 ID (CONVERSATION_ID)，多用户应用要保证唯一。

4. 默认使用内存存储 + MessageWindowChatMemory, 窗口默认 20 条。
5. 窗口按整轮 (turn) 淘汰, 不会从中间截断, SystemMessage 始终保留。
6. JDBC 持久化不支持工具调用消息, 且 2.0.0 起新增 sequence_id 列。
7. 生产/多实例环境建议使用持久化存储库, 避免使用内存实现。

(完)